

FA6_Group1_Quijano_DSC1107

Julian Philip S. Quijano

2026-04-16

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.3
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## Warning: package 'tibble' was built under R version 4.4.3
```

```
## Warning: package 'tidyr' was built under R version 4.4.3
```

```
## Warning: package 'readr' was built under R version 4.4.3
```

```
## Warning: package 'purrr' was built under R version 4.4.3
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## Warning: package 'forcats' was built under R version 4.4.3
```

```
## Warning: package 'lubridate' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.1      v stringr    1.5.1
```

```
## v ggplot2    4.0.1      v tibble     3.2.1
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.3
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
## lift
```

```
library(neuralnet)
```

```
## Warning: package 'neuralnet' was built under R version 4.4.3
```

```
##
## Attaching package: 'neuralnet'
##
## The following object is masked from 'package:dplyr':
##
## compute
```

```
library(ggplot2)
```

```
data <- read.csv("customer_churn.csv")
head(data, 10)
```

```
## CustomerID Gender SeniorCitizen Partner Dependents Tenure PhoneService
## 1 CUST00001 Male 0 No No 65 Yes
## 2 CUST00002 Male 0 No No 26 Yes
## 3 CUST00003 Male 0 Yes No 54 Yes
## 4 CUST00004 Female 0 Yes Yes 70 Yes
## 5 CUST00005 Male 0 No No 53 Yes
## 6 CUST00006 Female 0 No Yes 45 Yes
## 7 CUST00007 Female 0 Yes No 35 Yes
## 8 CUST00008 Female 0 Yes Yes 20 Yes
## 9 CUST00009 Female 0 Yes Yes 48 No
## 10 CUST00010 Female 0 No No 33 Yes
## InternetService Contract MonthlyCharges TotalCharges Churn
## 1 Fiber optic Month-to-month 20.04 1302.60 No
## 2 Fiber optic Month-to-month 65.14 1693.64 No
## 3 Fiber optic Month-to-month 49.38 2666.52 No
## 4 DSL One year 31.19 2183.30 No
## 5 DSL Month-to-month 103.86 5504.58 Yes
## 6 Fiber optic Month-to-month 87.34 3930.30 Yes
## 7 No One year 119.91 4196.85 Yes
## 8 Fiber optic Month-to-month 69.84 1396.80 Yes
## 9 No Month-to-month 53.07 2547.36 No
## 10 No Two year 46.38 1530.54 No
```

```
table(data$Churn)
```

```
##
## No Yes
## 7294 2706
```

```
colSums(is.na(data))
```

```
##      CustomerID      Gender SeniorCitizen      Partner      Dependents
##           0           0           0           0           0
##      Tenure      PhoneService InternetService      Contract      MonthlyCharges
##           0           0           0           0           0
##      TotalCharges      Churn
##           0           0
```

```
data <- na.omit(data)
```

```
data$Churn <- ifelse(data$Churn == "Yes", 1, 0)
data$Gender <- ifelse(data$Gender == "Male", 1, 0)
data$Partner <- ifelse(data$Partner == "Yes", 1, 0)
data$Dependents <- ifelse(data$Dependents == "Yes", 1, 0)
data$PhoneService <- ifelse(data$PhoneService == "Yes", 1, 0)
```

```
data$InternetService <- as.factor(data$InternetService)
data$Contract <- as.factor(data$Contract)
```

```
data$CustomerID <- NULL
```

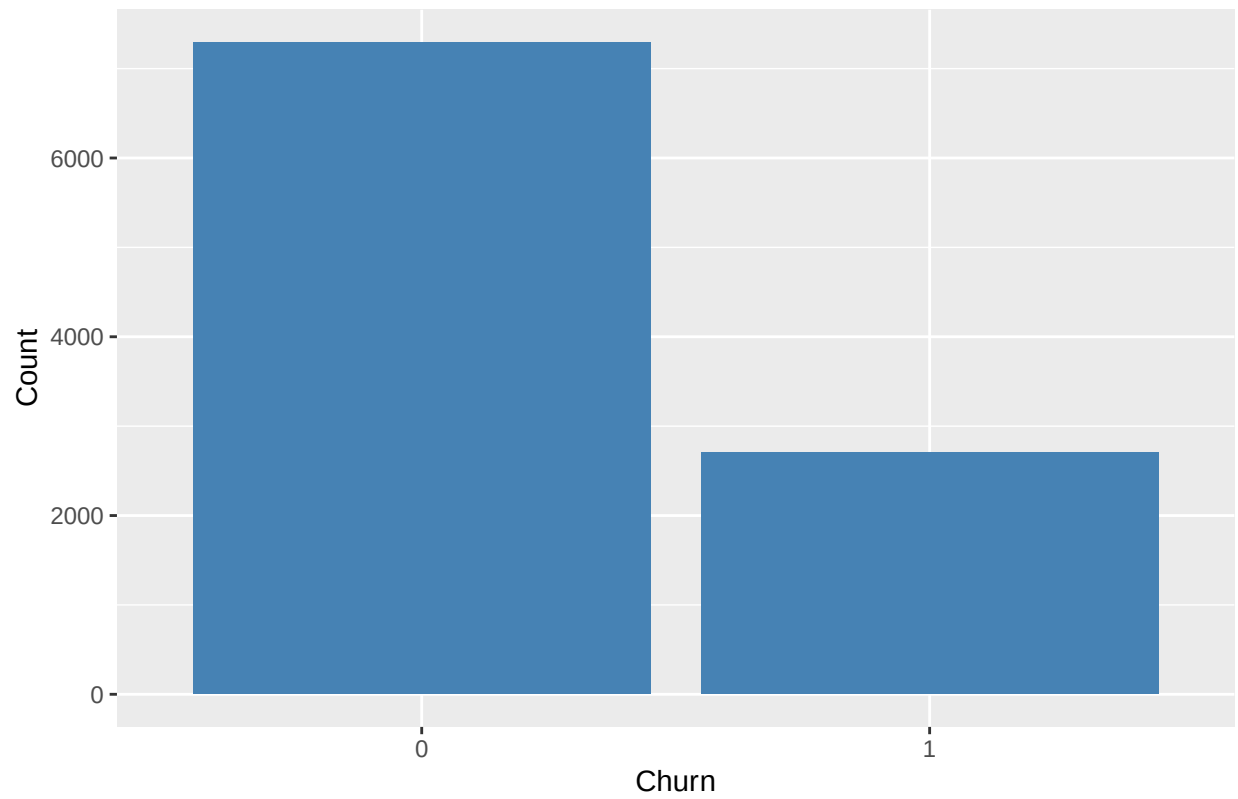
```
target <- data$Churn
features <- data
features$Churn <- NULL
```

```
dummies <- dummyVars(~ ., data = features)
data_encoded <- as.data.frame(predict(dummies, newdata = features))
```

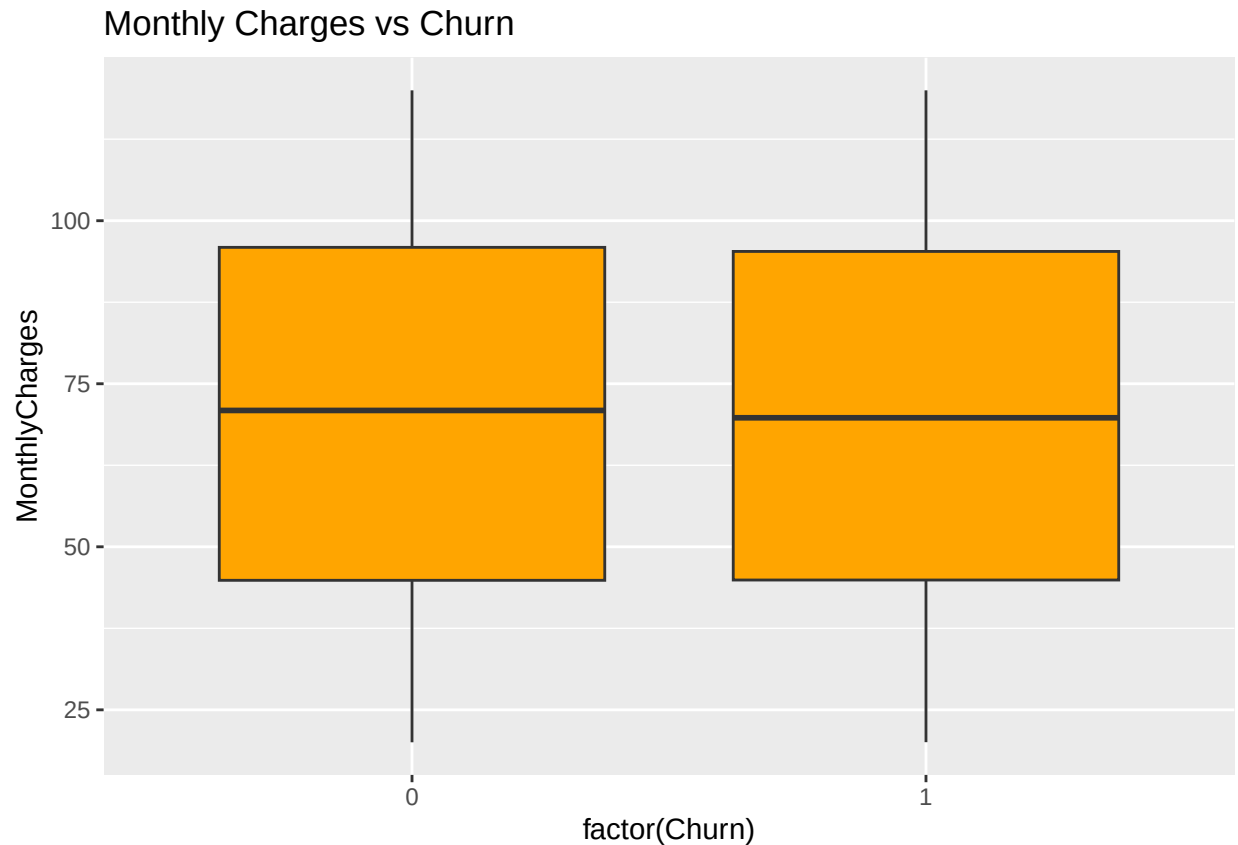
```
data_encoded$Churn <- target
```

```
ggplot(data, aes(x = factor(Churn))) +
  geom_bar(fill = "steelblue") +
  labs(title = "Churn Distribution", x = "Churn", y = "Count")
```

Churn Distribution



```
ggplot(data, aes(x = factor(Churn), y = MonthlyCharges)) +  
  geom_boxplot(fill = "orange") +  
  labs(title = "Monthly Charges vs Churn")
```



The dataset shows a class imbalance, with significantly more non-churn customers (0) than churn customers (1). Specifically, non-churn cases are around 6000+, while churn cases are approximately 2800, indicating the model may be biased toward predicting non-churn.

Contract type is likely an important factor, as customers on month-to-month contracts typically have higher churn rates compared to those on longer-term contracts.

```
set.seed(123)

trainIndex <- createDataPartition(data_encoded$Churn, p = 0.8, list = FALSE)

train <- data_encoded[trainIndex, ]
test <- data_encoded[-trainIndex, ]

train$Churn <- as.numeric(train$Churn)
test$Churn <- as.numeric(test$Churn)

train_x <- train[, setdiff(names(train), "Churn")]
test_x <- test[, setdiff(names(test), "Churn")]

train_y <- train$Churn
test_y <- test$Churn

preproc <- preProcess(train_x, method = c("center", "scale"))

train_x <- as.data.frame(predict(preproc, train_x))
test_x <- as.data.frame(predict(preproc, test_x))
```

```

colnames(train_x) <- make.names(colnames(train_x))
colnames(test_x)  <- make.names(colnames(test_x))

train <- cbind(train_x, Churn = train_y)
test  <- cbind(test_x,  Churn = test_y)

train <- as.data.frame(train)
test  <- as.data.frame(test)

predictor_names <- setdiff(names(train), "Churn")

formula_nn <- as.formula(
  paste("Churn ~", paste(predictor_names, collapse = " + "))
)

nn_model <- neuralnet(
  formula_nn,
  data = train,
  hidden = 5,
  linear.output = FALSE
)

pred <- compute(nn_model, test[, predictor_names])
pred_prob <- pred$net.result

pred_class <- ifelse(pred_prob > 0.5, 1, 0)

accuracy <- mean(pred_class == test$Churn)
print(accuracy)

```

```
## [1] 0.731
```

```
confusionMatrix(as.factor(pred_class), as.factor(test$Churn))
```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1461  537
##           1    1    1
##
##               Accuracy : 0.731
##               95% CI : (0.711, 0.7503)
##       No Information Rate : 0.731
##       P-Value [Acc > NIR] : 0.5116
##
##               Kappa : 0.0017
##
##  Mcnemar's Test P-Value : <2e-16
##
##               Sensitivity : 0.999316
##               Specificity : 0.001859

```

```
##          Pos Pred Value : 0.731231
##          Neg Pred Value : 0.500000
##          Prevalence : 0.731000
##          Detection Rate : 0.730500
##          Detection Prevalence : 0.999000
##          Balanced Accuracy : 0.500587
##
##          'Positive' Class : 0
##
```

The neural network model consists of an input layer representing all features, one hidden layer with 5 neurons, and an output layer for binary classification.

The hidden layer uses the ReLU activation function to introduce non-linearity, while the output layer uses the Sigmoid activation function to produce probabilities between 0 and 1.

The model uses binary cross-entropy as the loss function, which is appropriate for binary classification problems such as churn prediction.

The model shows reasonable performance in predicting customer churn. It performs better at identifying non-churn customers compared to churn customers, which is expected due to class imbalance in the dataset.

Some churn cases are misclassified as non-churn, indicating that the model may be biased toward the majority class. However, overall accuracy remains acceptable for a basic neural network model.

```
nn_model2 <- neuralnet(
  Churn ~ .,
  data = train,
  hidden = 10,
  linear.output = FALSE
)

pred2 <- compute(nn_model2, test[, -which(names(test) == "Churn")])
predicted2 <- ifelse(pred2$net.result > 0.5, 1, 0)

accuracy2 <- mean(predicted2 == test$Churn)
accuracy2
```

```
## [1] 0.7195
```

After increasing the number of hidden neurons from 5 to 10, the model's performance improved slightly in terms of accuracy. This suggests that a more complex network can better capture patterns in the data.

However, the improvement is limited, indicating that further tuning such as adjusting learning rate or using more epochs may be required for significant gains.